

Course Code: CS1105		
Name of the Programme: BCA (Hons.)		
Name of the Course: Introduction to Version Control		
Semester: II		
Course Credit	No. of hours per week (L+T+P)	Total no. of Teaching Hours
2	0:1:2	45
Course Objectives:		
a) Develop computational thinking and effective problem-solving abilities		
b) Gain hands-on experience with version control, AI tools, and modern technologies.		
c) Encourage collaborative learning and peer-to-peer interaction.		
d) Practice ethical, responsible, and reflective coding habits.		

Course outline (Syllabus of the course)

Module - 1 (Introduction to Coding Platforms and Tools)	No. of Hours
	3 Hrs.
Overview of Coding Platforms: LeetCode, Codeforces, HackerRank, CodeChef, GeeksforGeeks. Benefits of practicing on coding platforms, Understanding problem-solving and competitive programming.	
Module - 2 (Introduction to Git and Version Control)	No. of Hours
	3 Hrs.
Version Control Basics: What is version control, Advantages of version control systems (VCS), Types of VCS: Centralized vs. Distributed. Introduction to Git: What is Git, Installing Git on Windows/Linux/Mac, Git vs other version control tools.	
Module - 3 (Working with Repositories)	No. of Hours
	3 Hrs.
Git Repositories: Understanding local and remote repositories, Hosting platforms: GitHub, GitLab, Bitbucket, Creating and managing repositories on GitHub. Collaborating with Git: Forking and cloning repositories, pull requests and merge conflicts, Understanding. gitignore and branching.	
Module - 4 (Co-Pilot and AI-Assisted Coding)	No. of Hours
	3 Hrs.
Introduction to GitHub Co-Pilot: What is Co-Pilot, Role of AI in coding and productivity using GitHub Co-Pilot. Setting up Co-Pilot in Visual Studio Code, Writing efficient code with AI assistance, Ethical considerations and limitations of AI-assisted coding.	

Module - 5 (Advanced Version Control and Best Practices)	No. of Hours
	3 Hrs.
Advanced Git Concepts: Stashing changes (git stash), Rebasing vs merging, Cherry-picking commits, Tagging versions in Git. Version Control Best Practices: Writing meaningful commit messages, Structuring repositories effectively, Using branches for feature development.	

Course Outcomes:
a) Understand the purpose, structure, and benefits of centralized and distributed version control systems.
b) Develop problem-solving abilities using coding platforms and real-world programming tasks.
c) Demonstrate proficiency in Git and GitHub for collaborative version control and project management.
d) Apply advanced Git techniques and adopt best practices to enhance workflow efficiency and team productivity.

TextBook:
1 Chacon, S., & Straub, B. (2014). <i>Pro Git</i> (2nd ed.). Apress. ISBN: 978-1484200773

Reference Books:
1 Loeliger, J., & McCullough, M. (2012). <i>Version control with Git</i> (2nd ed.). O'Reilly Media. ISBN: 978-1449316389
2 Skoulirikari, A. (2021). <i>Learning Git: A hands-on and visual guide to the basics of Git</i> . O'Reilly Media. ISBN: 978-1492092513

Tutorials
1 Introduction to competitive programming and coding platforms: LeetCode, HackerRank, Codeforces
2 Benefits of practicing on coding platforms; Problem-solving strategies
3 Introduction to Version Control Systems (VCS): Centralized vs. Distributed
4 Installing Git on Windows/Linux/Mac; Setting up user credentials
5 Basic Git operations: git init, git add, git commit, git status, git log
6 Understanding local vs. remote repositories; Git hosting platforms (GitHub, GitLab, Bitbucket)
7 Creating and linking repositories on GitHub; Pushing and pulling changes
8 Forking and cloning repositories; Understanding gitignore
9 Branching in Git: Creating, switching, and deleting branches
10 Merging branches; Identifying and resolving merge conflicts
11 Introduction to GitHub collaboration: Pull requests and code reviews
12 Best practices in reviewing and managing pull requests

13	Advanced Git features: git stash, git rebase vs git merge, git tag, git cherry-pick
14	Introduction to GitHub Co-Pilot; Installing and configuring Co-Pilot in VS Code
15	Ethical use of AI in coding; Best practices: commit messages, branching strategies, clean history

Laboratory Components	
1	Create accounts on LeetCode, HackerRank, and Codeforces and solve two problems on each platform.
2	Focus on array and string problems on LeetCode and reflect on solution strategies.
3	Participate in a Codeforces contest and submit a solution analysis report.
4	Install Git and configure user credentials on your system.
5	Initialize a new Git repository and commit a sample file using basic Git commands.
6	View repository status and commit history using git status and git log.
7	Create and link a local repository with GitHub and push commits using git push.
8	Pull updates from a remote repository and manage repository synchronization.
9	Fork and clone a GitHub repository, create a pull request with changes.
10	Create feature branches, switch between them, and merge them using Git.
11	Simulate and resolve a merge conflict in a collaborative project setup.
12	Use git stash to save and apply temporary changes during development.
13	Rebase a feature branch onto the main branch using git rebase
14	Tag specific commits using git tag and cherry-pick selected commits into another branch.
15	Set up GitHub Co-Pilot in VS Code and generate code suggestions for simple functions.